The Numerov Method — Solutions to the Exercises

Worked solutions with predictions, computations, tables, convergence plots and discussion (companion to the course notes)

November 2026

Table of Contents

How the solutions are organised 3

Exercise 1 — Order from two grids 4

Exercise 2 — A better start 5

Exercise 3 — Points per wavelength 6

Exercise 4 — Dispersion relation in the scattering problem 8

Exercise 5 — Thick barrier 10

Exercise 6 — Smooth barrier 12

Exercise 7 — One-sided shooting 14

Exercise 8 — Anharmonic oscillator 16

Exercise 9 — Finite square well 18

Exercise 10 — Round-off in single precision 20

Appendix — Codes 22

- Exercises 1–5 (Python) 22
- Exercises 6–9 (Python) 25
- Exercise 10 (Fortran 90, single and double precision) 29
- Exercise 10 — plotting script 30

How the solutions are organised

Each solution follows the same pattern: **(1) the question**, **(2) a prediction** made from the theory of Part I of the course *before* computing, **(3) the method** (what was changed in the course codes), **(4) the results**, and **(5) the answer and its explanation**. All numbers were produced by the programs listed in the Appendix (`ex01_05.py`, `ex06_09.py`, `ex10_precision.F90`, `ex10_plot.py`). Units $\hbar = m = 1$ throughout.

Ex.	Topic	Short answer
1	Order from two grids	$p \approx 4.00$; the 17.6 (p = 4.14) is a round-off artefact
2	Third-order Taylor start	Start error $O(h^5)$ ⇒ global order 4 restored (measured 4.00)
3	Points per wavelength	$N_{\min} = 254$ ⇒ 51 points per wavelength for 10^{-6}; error $= (kh)^4/240$
4	Discrete plane waves $e^{ik_h x}$	Error falls by only 0–8 %: the dominant error is in the barrier, and T ignores phases
5	Thick barrier	$T = 4.6537 \times 10^{-15}$, WKB too small by the prefactor 2.56; $R + T$ cannot resolve T
6	Smooth barrier	No correction needed: order 3.99–4.02, accuracy 10^{-12}
7	One-sided shooting	E_0 still converges for $5 \lesssim L \lesssim 20$, but the tail of ψ is destroyed and $L \gtrsim 27$ overflows
8	Anharmonic oscillator	$\Delta E_n/\lambda \to 3/4\,(2n^2 + 2n + 1)$ as $\lambda \to 0$; at $\lambda = 0.1$, $E_0 = 0.5591463271$
9	Finite square well	3 bound states, exact conditions satisfied; corrected scheme order 3.90, naive order 2
10	Single precision	Optimum $h \approx 0.125$, best error 7.6×10^{-6} (double: $h \approx 0.008$, 1.1×10^{-11})

Exercise 1 — Order from two grids

Question. Using the Problem 1, Part A table, compute $p = \log_2[\varepsilon(h)/\varepsilon(h/2)]$ for every pair of rows and explain the last value (17.6).

Prediction. For a fourth-order method $\varepsilon(h) \approx Ch^4$, so $\varepsilon(h)/\varepsilon(h/2) = 16$ and $p = 4$, as long as round-off is negligible.

Results (the table was extended by two more halvings to show what happens next):

N → 2N	ε(h)	ε(h/2)	ratio	p
25 → 50	1.554e-02	1.011e-03	15.4	3.94
50 → 100	1.011e-03	6.285e-05	16.1	4.01
100 → 200	6.285e-05	3.924e-06	16.0	4.00
200 → 400	3.924e-06	2.455e-07	16.0	4.00
400 → 800	2.455e-07	1.533e-08	16.0	4.00
800 → 1600	1.533e-08	9.503e-10	16.1	4.01
1600 → 3200	9.503e-10	5.401e-11	**17.6**	**4.14**
3200 → 6400	5.401e-11	2.180e-10	0.25	−2.01
6400 → 12800	2.180e-10	1.509e-09	0.14	−2.79

Explanation of 17.6.

- The first pair gives 3.94 because $h = 0.4$ ($kh = 0.8$) is not yet in the asymptotic regime: higher-order terms h^6, h^8 still contribute.
- From $N = 50$ to $N = 1600$ the value is 4.00 ± 0.01 — the theoretical order.
- Extrapolating the clean h^4 law from $N = 200$ predicts $\varepsilon(N = 3200) = 5.99 \times 10^{-11}$; the computed value is 5.40×10^{-11}. The measured error is *smaller* than the truncation error because the accumulated round-off error (of size 10^{-11} here, with a random sign) happens to partially cancel it. The ratio is therefore artificially large (17.6 instead of 16).
- One more halving confirms the diagnosis: the error *increases* ($p < 0$). Below $h \approx 3 \times 10^{-3}$ the round-off error, which grows roughly like ϵ/h^2 for a second-order recursion, dominates. A ratio larger than 16 is thus the first warning sign that the round-off floor has been reached; the optimum step for this problem is $h \approx 3 \times 10^{-3}$.

Exercise 2 — A better start

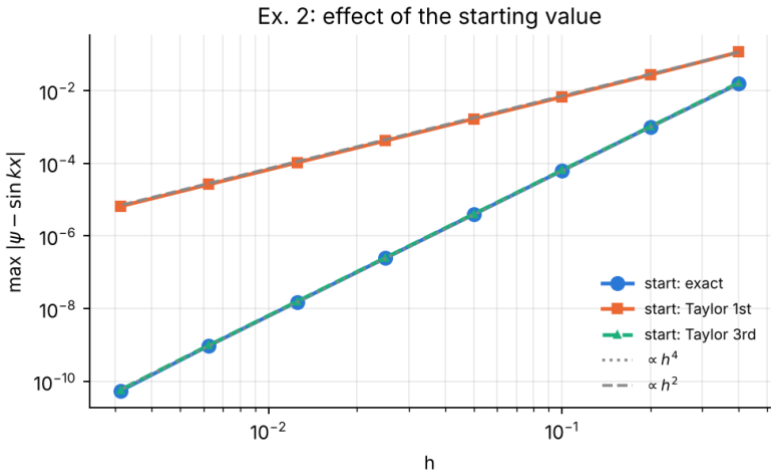
Question. Use $\psi_1 = kh - (kh)^3/6$. Predict, then measure, the global order.

Prediction. The exact value is $\sin kh = kh - (kh)^3/6 + (kh)^5/120 - \cdots$, so the start error is $\delta = (kh)^5/120 = O(h^5)$. From Part I, a start error δ produces a global error $O(\delta/h) = O(h^4)$ — the same order as Numerov's own truncation error. **Predicted global order: 4.** (The first-order start $\psi_1 = kh$ has $\delta = O(h^3)$ and gives order 2.)

Results ($k = 2, X = 10$):

N	h	error, exact start	error, Taylor 1st	error, Taylor 3rd	ratio (3rd)
25	0.400	1.554e-02	1.144e-01	1.647e-02	—
50	0.200	1.011e-03	2.711e-02	1.021e-03	16.1
100	0.100	6.285e-05	6.694e-03	6.430e-05	15.9
200	0.050	3.924e-06	1.668e-03	4.014e-06	16.0
400	0.025	2.455e-07	4.168e-04	2.510e-07	16.0
800	0.0125	1.533e-08	1.042e-04	1.567e-08	16.0
1600	0.00625	9.503e-10	2.604e-05	9.716e-10	16.1
3200	0.003125	5.401e-11	6.510e-06	5.770e-11	16.8

Fitted orders: exact start 3.99, first-order Taylor 2.02, **third-order Taylor 4.00**.



Exercise 2. The third-order Taylor start (triangles, dashed) lies on top of the exact start: the global order 4 is restored.

Answer. As predicted, the global order is 4. The third-order start adds only about 2 % to the error (the start contribution $|\varepsilon_3 - \varepsilon_{\text{exact}}|$ is 1.4×10^{-6} at $N = 100$ against a total of 6.4×10^{-5}). Rule: *the starting value must be accurate to* $O(h^5)$; any Taylor start of order 4 or higher is sufficient.

Exercise 3 — Points per wavelength

Question. For the box ($L = 1$), find the smallest N that gives E_{10} with relative error below 10^{-6}; express it as points per wavelength and compare with the rule of thumb.

Theory (prediction). In the box, $\sin(k_h x)$ with the *discrete* wave number k_h satisfies the Numerov recursion exactly and vanishes at $x = 0$ and $x = L$ when $k_h = n\pi/L$. Hence the numerical eigenvalue obeys $k_h(E) = n\pi/L$ exactly. Using the dispersion relation of Part I, $k_h = k[1 + (kh)^4/480]$, the numerical k is too small by the factor $1 - (kh)^4/480$ and, since $E \propto k^2$,

$$\frac{E_n^{\text{num}} - E_n}{E_n} \approx -\frac{(kh)^4}{240}, \qquad kh = \frac{2\pi}{P},$$

where $P = \lambda/h$ is the number of grid points per wavelength. Setting this equal to 10^{-6}:

$$P = \frac{2\pi}{(240 \times 10^{-6})^{1/4}} = \frac{2\pi}{0.1245} \approx 50.5.$$

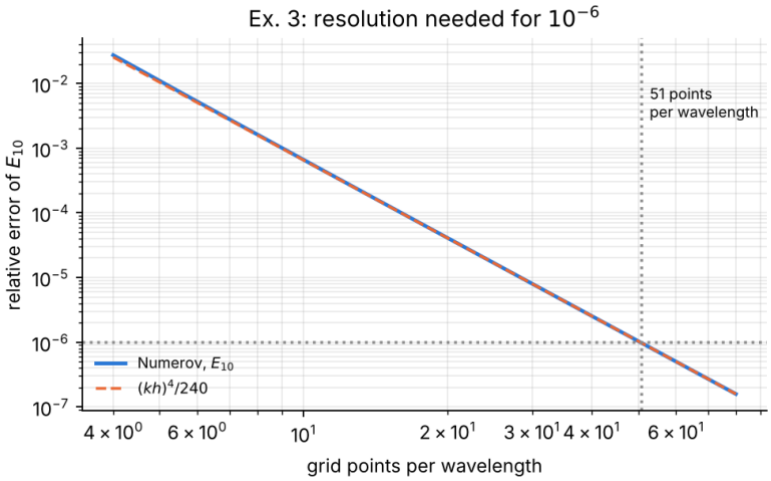
The numerical energy is always **below** the exact one (as in the table of Problem 1).

Results. For $n = 10$ the wavelength is $\lambda = 2L/n = 0.2$.

N	kh	relative error of E_{10}	$(kh)^4/240$
40	0.785	1.622e-03	1.585e-03
100	0.314	4.074e-05	4.059e-05
200	0.157	2.539e-06	2.537e-06
252	0.125	1.007e-06	—
254	**0.124**	**9.757e-07**	9.751e-07

$N_{\min} = 254$, i.e. $h = 0.00394$, **50.8 points per wavelength**. The same criterion is independent of n, as the theory says:

n	N_min	points per wavelength
1	26	52.0
2	52	52.0
5	128	51.2
10	254	50.8
20	506	50.6



Exercise 3. Relative error of E_{10} versus points per wavelength; the formula $(kh)^4/240$ (dashed) coincides with the computation.

Comparison with the rule of thumb. The rule "20–30 points per wavelength" of Part I gives relative errors of $(2\pi/20)^4/240 = 4.1 \times 10^{-5}$ to $(2\pi/30)^4/240 = 8.0 \times 10^{-6}$: it is adequate for 4–5 significant digits. For 10^{-6} one needs about **51 points per wavelength**; in general $P \approx 2\pi\,(240\,\varepsilon)^{-1/4}$ for a target relative error ε — doubling the accuracy demand costs only a factor $2^{1/4} = 1.19$ in grid points, the benefit of a fourth-order method.

Exercise 4 — Dispersion relation in the scattering problem

Question. Replace e^{ikx} by $e^{ik_h x}$ in the starting values and in the extraction of A, B (Problem 2). Does the error decrease? Why?

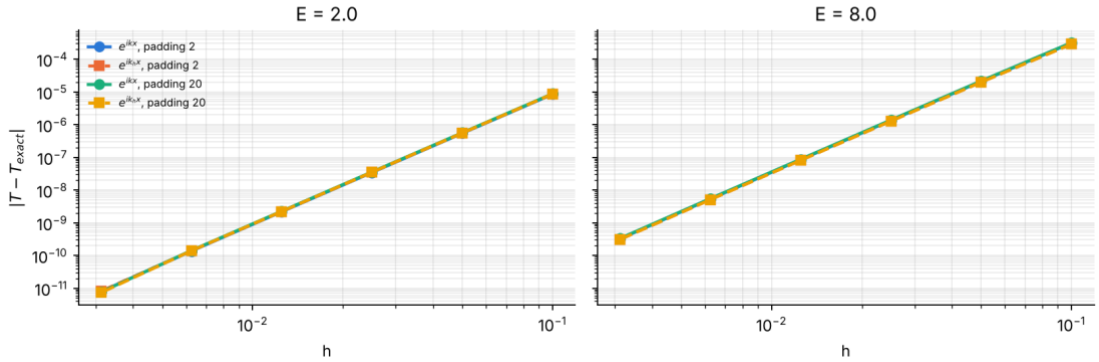
Prediction. In the free regions $V = 0$ the *discrete* solutions of the Numerov recursion are exactly $e^{\pm ik_h x}$. With $e^{ik_h x}$ the right boundary is an exact transmitted discrete wave and the decomposition on the left is exact, so the free regions contribute *no* error and the result should not depend on the padding length. With e^{ikx} the two starting values contain a tiny admixture of the reflected discrete wave, of relative size $\sim (k_h - k)/k \sim (kh)^4/480$. Two arguments suggest that the gain will be small: (i) $T = 1/|A|^2$ depends only on moduli, so the accumulated phase error $k^5 h^4 X/480$ cancels; (ii) the $(kh)^4/480$ coefficient is small compared with the error committed inside the barrier.

Method. `scatter(..., use_kh=True)` computes $k_h = \arccos[(1 - 5/12\,(kh)^2)/(1 + 1/12\,(kh)^2)]/h$ and uses it in both the start and the 2×2 extraction. The corrected discontinuity treatment of Problem 2 is used in all runs.

Results ($V_0 = 5$, $a = 1$):

h	E=2, pad 2: e^{ikx}	E=2, pad 2: $e^{ik_h x}$	E=8, pad 2: e^{ikx}	E=8, pad 2: $e^{ik_h x}$	E=8, pad 20: e^{ikx}	E=8, pad 20: $e^{ik_h x}$
0.1	8.41e-06	8.59e-06	3.12e-04	2.89e-04	3.19e-04	2.89e-04
0.05	5.40e-07	5.52e-07	2.10e-05	1.98e-05	2.16e-05	1.98e-05
0.025	3.43e-08	3.50e-08	1.34e-06	1.28e-06	1.38e-06	1.28e-06
0.0125	2.16e-09	2.20e-09	8.45e-08	8.09e-08	8.71e-08	8.09e-08
0.00625	1.36e-10	1.39e-10	5.29e-09	5.08e-09	5.46e-09	5.08e-09

Ex. 4: continuum vs discrete plane waves at the boundaries



Exercise 4. The four curves for each energy practically coincide: replacing k by k_h does not change the convergence.

Answer. The error changes by only −8 % to +2 %, and the order remains 4. With $e^{ik_h x}$ the result becomes **exactly independent of the padding length** (identical digits for padding 2 and 20), which confirms that the free regions are now treated exactly; with e^{ikx} a 2–10 % padding dependence remains. The error does not decrease substantially because:

1. T **is phase-insensitive.** The dominant consequence of $k \neq k_h$ is a phase error, which cancels in $|A|^2$ and $|B|^2$.
2. **The error lives in the barrier.** The remaining $O(h^4)$ error comes from integrating through the region $0 < x < a$, where ψ changes rapidly (evanescent or short-wavelength) and from the edge treatment — the boundary treatment cannot remove it.

The modification is still good practice: it makes the result independent of where the computational box ends.

Exercise 5 — Thick barrier

Question. Compute T for $V_0 = 5$, $a = 6$, $E = 1$; compare with the WKB estimate $T \approx e^{-2\kappa a}$. What limits the accuracy, and what happens to $R + T$?

Analytical values. $\kappa = \sqrt{2(V_0 - E)} = 2.82843$, $\kappa a = 16.9706$.

Quantity	Value
Exact T	4.653679e-15
WKB $e^{-2\kappa a}$	1.817843e-15
Thick-barrier limit $\frac{16E(V_0-E)}{V_0^2} e^{-2\kappa a}$	4.653679e-15
Ratio exact/WKB	2.560

For $\kappa a \gg 1$, $\sinh^2(\kappa a) \approx e^{2\kappa a}/4$ and the exact formula reduces to $T \approx \frac{16E(V_0-E)}{V_0^2} e^{-2\kappa a}$. The simple WKB exponent captures the **exponential dependence** perfectly but misses the **prefactor** $16E(V_0 - E)/V_0^2 = 2.56$, which comes from the abrupt (non-WKB) edges of the barrier. The ratio WKB/exact = 0.391 is the same for every thickness $a \geq 1$ (table below).

Numerov results (corrected edge scheme):

h	T Numerov	relative error	R	R+T−1
0.05	4.65345430e-15	4.8e-05	0.99999999999999079	−4.6e-15
0.025	4.65366495e-15	3.1e-06	0.99999999999996469	−3.1e-14
0.0125	4.65367840e-15	2.0e-07	0.99999999999993749	−5.8e-14
0.00625	4.65367925e-15	1.2e-08	1.00000000000018940	+1.9e-13
0.003125	4.65367931e-15	6.0e-10	0.99999999999941325	−5.8e-13

Thickness scan at $h = 0.00625$:

a	T Numerov	T exact	relative error	WKB/exact	R+T−1
1	8.9258e-03	8.9258e-03	6.5e-09	0.391	−3.8e-11
2	3.1243e-05	3.1243e-05	7.6e-09	0.391	−2.8e-13
4	3.8131e-10	3.8131e-10	9.9e-09	0.391	−1.4e-15
6	4.6537e-15	4.6537e-15	1.2e-08	0.391	1.9e-13
8	5.6796e-20	5.6796e-20	1.5e-08	0.391	−4.7e-14
12	8.4597e-30	8.4597e-30	1.9e-08	0.391	−5.0e-13

What limits the accuracy?

- T itself is obtained with a *relative* accuracy of 10^{-8} even when $T \sim 10^{-30}$. This works because the **backward** integration goes in the stable direction: going from right to left through the barrier the physical solution *grows* like $e^{\kappa(a-x)}$, so round-off errors are swamped, and $T = 1/|A|^2$ is computed directly from a large number ($|A| = 1.5 \times 10^7$) — never as a small difference.
- The remaining error is the Numerov discretisation error (order 4 in the table), whose relative size grows slowly (linearly) with a because the phase/decay-rate error accumulates over the barrier: 6.5×10^{-9} at $a = 1$ to 1.9×10^{-8} at $a = 12$.
- The ultimate limit is overflow: $|A| \sim e^{\kappa a}$ exceeds 10^{308} when $\kappa a > 709$, i.e. $a \gtrsim 250$ here.

What happens to $R + T$? $R = 1 - 4.65 \times 10^{-15}$ cannot be represented more precisely than about $\pm 10^{-16}$ in double precision, and its round-off error (10^{-13}–10^{-12}: the relative round-off of $|A|$ and $|B|$, accumulated over thousands of steps) is **larger than T itself**. $R + T - 1$ therefore shows only round-off noise and contains no information about T. Conclusion: for opaque barriers *never* compute T as $1 - R$ and never use $R + T = 1$ as an accuracy test; compute T from the transmitted amplitude.

Exercise 6 — Smooth barrier

Question. Replace the square barrier by $V(x) = V_0/\cosh^2(x/w)$, whose exact T is known; show that no correction is needed to obtain fourth order.

Exact result (Pöschl–Teller / Eckart barrier; Landau & Lifshitz §25) for $\hbar = m = 1$ and $8V_0w^2 > 1$:

$$T(E) = \frac{\sinh^2(\pi k w)}{\sinh^2(\pi k w) + \cosh^2\left(\frac{\pi}{2}\sqrt{8V_0w^2 - 1}\right)}, \qquad k = \sqrt{2E},$$

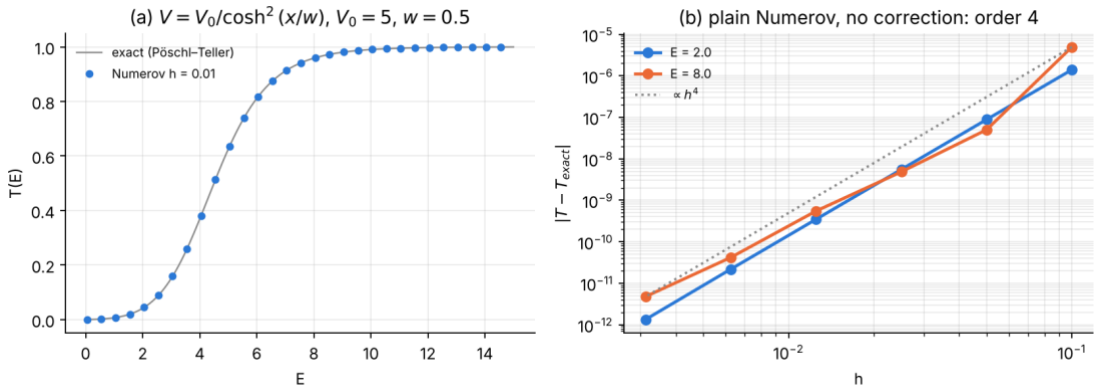
($\cosh^2 \to \cos^2\left(\frac{\pi}{2}\sqrt{1 - 8V_0w^2}\right)$ when $8V_0w^2 < 1$). Parameters: $V_0 = 5$, $w = 0.5$ ($8V_0w^2 = 10$), computational box $[-12,12]$, where $V(\pm 12) = 3 \times 10^{-20}$ is negligible.

Prediction. V is analytic, so ψ and all its derivatives are continuous; the Taylor expansions behind Numerov hold everywhere and the plain scheme should be fourth order with no special treatment.

Results. Plain Numerov, no edge nodes, no correction:

E	T Numerov (h = 0.01)	T exact	|ΔT|	R+T−1
1	0.0066549324	0.0066549324	3.6e-12	8.2e-13
3	0.1505171785	0.1505171781	3.7e-10	1.8e-13
5	0.6247014391	0.6247014405	1.4e-09	1.8e-13
8	0.9585694821	0.9585694818	2.5e-10	2.6e-13

h	|ΔT|, E = 2	ratio	|ΔT|, E = 8	ratio
0.1	1.40e-06	—	4.95e-06	—
0.05	8.83e-08	15.9	5.02e-08	98.6
0.025	5.54e-09	15.9	4.85e-09	10.4
0.0125	3.47e-10	16.0	5.49e-10	8.8
0.00625	2.18e-11	15.9	4.17e-11	13.2
0.003125	1.33e-12	16.4	4.74e-12	8.8
order	**3.99**		**4.02**	



Exercise 6. (a) Transmission through the smooth barrier; (b) plain Numerov converges with slope 4 without any correction.

Answer. The fitted orders are 3.99 and 4.02: **no correction is needed**. Compare with the square barrier, where the uncorrected scheme was second order. For $E = 8$ the individual ratios scatter (98.6, 10.4, …) because the error is a sum of several h^4 contributions of different sign that partly cancel at some h; the

overall slope is still 4. The maximum error over $0 < E < 15$ at $h = 0.01$ is 2.6×10^{-9}. Lesson: the loss of order in Problem 2 was caused by the *discontinuity of the potential*, not by the scattering formulation.

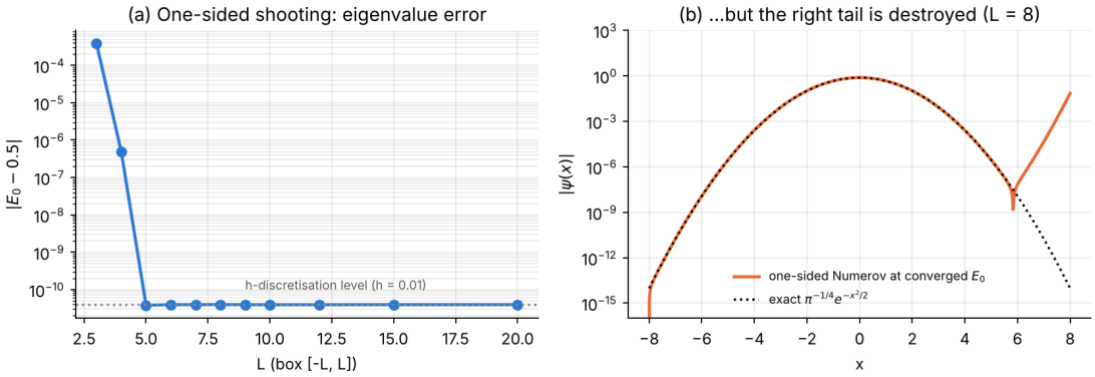
Exercise 7 — One-sided shooting

Question. Modify Problem 3 to integrate from $-L$ to $+L$ and require $\psi_N = 0$. For which L does the bisection still converge to E_0? Explain with fault no. 4.

Method. $\psi_0 = 0$, $\psi_1 = 10^{-10}$, Numerov from $-L$ to $+L$ with $h = 0.01$; bisection on the sign of $F(E) = \psi_N(E)$ in $[0.3, 0.7]$ (100 halvings).

Results.

L	E_0 (one-sided)	$\lvert E_0 - 0.5 \rvert$	$e^{L^2/2}$
3	0.500391082888	3.9e-04	9.0e+01
4	0.500000490818	4.9e-07	3.0e+03
5	0.500000000038	3.8e-11	2.7e+05
6	0.499999999961	3.9e-11	6.6e+07
8	0.499999999961	3.9e-11	7.9e+13
10	0.499999999961	3.9e-11	5.2e+21
15	0.499999999961	3.9e-11	7.2e+48
20	0.499999999961	3.9e-11	7.2e+86
30	overflow (NaN)	—	—



Exercise 7. (a) The eigenvalue from one-sided shooting reaches the two-sided accuracy for $L \geq 5$. (b) The wavefunction at the converged energy, $L = 8$: the right tail departs from $e^{-x^2/2}$ at $x \approx 6$ and grows exponentially.

Answer and explanation. Perhaps surprisingly, the **eigenvalue** converges to E_0 for all $5 \lesssim L \lesssim 20$ with the same accuracy (3.9×10^{-11}) as the two-sided method. For $L < 5$ the error is the box-truncation error (identical to the two-sided result of Problem 3); for $L \gtrsim 27$ the computation overflows. What happens is:

1. Beyond the right turning point the computed solution is $\psi = \alpha(E)\,\phi_\downarrow + \beta(E)\,\phi_\uparrow$, with $\phi_\uparrow \sim e^{+x^2/2}$. At $x = L$ the growing part dominates completely, so $\mathrm{sign}\psi_N = \mathrm{sign}\beta(E)$. $\beta(E)$ changes sign exactly at the discrete eigenvalue, and round-off adds to β a contribution equivalent to an energy shift of order $\epsilon \approx 10^{-16}$. Bisection on the *sign* is therefore insensitive to the instability: the eigenvalue survives.

2. The **wavefunction does not survive** (fault no. 4). Even at the converged energy, the residual β (from the finite bisection resolution and round-off) is amplified by $e^{x^2/2}$. Panel (b) shows that for $L = 8$ the computed ψ follows the exact Gaussian only up to $x \approx 6$ and then grows to $|\psi(L)| \approx 0.07$ — a value 10^{12} times larger than the exact $\psi(8) \approx 10^{-14}$. Such a function cannot be normalised reliably, and expectation values ($\langle x^2 \rangle$, the virial check) are wrong.

3. **Overflow.** The growing solution reaches $\sim 10^{-10} e^{L^2}$ at $x = +L$; this exceeds the largest double (1.8×10^{308}) when $L^2 \gtrsim 732$, i.e. $L \gtrsim 27$ (NaN at $L = 30$).

So one-sided shooting gives correct energies for moderate L but wrong wavefunctions and fails for large boxes; the two-sided method of Problem 3 avoids both problems by never integrating into a forbidden region.

Exercise 8 — Anharmonic oscillator

Question. Use the Problem 3 code for $V = x^2/2 + \lambda x^4$, $\lambda = 0.1$; check against first-order perturbation theory $\Delta E_n = \frac{3\lambda}{4}(2n^2 + 2n + 1)$ for small λ.

First-order perturbation theory. With $x = (a + a^\dagger)/\sqrt{2}$, $\langle n|x^4|n\rangle = \frac{3}{4}(2n^2 + 2n + 1)$, hence $\Delta E_n^{(1)} = \frac{3\lambda}{4}(2n^2 + 2n + 1)$: coefficients 0.75, 3.75, 9.75 for $n = 0,1,2$. For the ground state the series continues $E_0 = \frac{1}{2} + \frac{3}{4}\lambda - \frac{21}{8}\lambda^2 + \frac{333}{16}\lambda^3 - \cdots$ (an asymptotic, divergent series).

Method. The Problem 3 module is imported unchanged and only its potential is replaced (`P3.V = lambda x: 0.5*x*x + lam*x**4`); box $[-6,6]$, $h = 0.01$. As an **independent check**, the Hamiltonian is also diagonalised in a basis of 160 harmonic-oscillator states (`ho_basis_energies`).

Results.

λ	n	E Numerov	E basis	(E − E⁽⁰⁾)/λ	1st-order coef.
1e-4	0	0.500074973732	0.500074973771	0.749737	0.75
1e-4	1	1.500374793721	1.500374793994	3.747937	3.75
1e-4	2	2.500974231527	2.500974232502	9.742315	9.75
1e-3	0	0.500747395535	0.500747395575	0.747396	0.75
1e-3	1	1.503729615426	1.503729615703	3.729615	3.75
1e-3	2	2.509674351733	2.509674352726	9.674352	9.75
1e-2	0	0.507256204483	0.507256204525	0.725620	0.75
1e-2	1	1.535648277992	1.535648278297	3.564828	3.75
1e-2	2	2.590845795041	2.590845796191	9.084580	9.75
0.1	0	**0.559146327122**	0.559146327184	0.591463	0.75
0.1	1	**1.769502643379**	1.769502643950	2.695026	3.75
0.1	2	**3.138624305981**	3.138624308498	6.386243	9.75



Exercise 8. Energy shift divided by λ; the dotted lines are the first-order coefficients $\frac{3}{4}(2n^2 + 2n + 1)$, the dashed line the second-order result $\frac{3}{4} - \frac{21}{8}\lambda$ for $n = 0$.

Answer.

- **Small λ.** $\left(E_n - E_n^{(0)}\right)/\lambda$ tends to 0.75, 3.75, 9.75 as $\lambda \to 0$ (at $\lambda = 10^{-4}$: 0.7497, 3.7479, 9.7423). The deviation is linear in λ, as expected from the second-order term: for $n = 0$, $\frac{3}{4} - \frac{21}{8}(10^{-3}) = 0.747375$, and adding the third-order term $\frac{333}{16}(10^{-3})^2$ gives 0.747396 — exactly the computed value.
- **λ = 0.1.** $E_0 = 0.5591463271$, in agreement with the basis diagonalisation to 6×10^{-11} (the Numerov discretisation error at $h = 0.01$). First-order perturbation theory gives 0.575 (+2.8 %); for $n = 2$ it gives 3.475 instead of 3.1386 (+11 %), because the higher states extend to larger $|x|$ where λx^4 is no longer small. Adding more terms does not help much (second order 0.54875, third order 0.56956): the perturbation series is asymptotic.
- **Practical remarks.** The Problem 3 code works unchanged because the turning-point estimate $\sqrt{2E}$ lies slightly outside the true turning point, which only places the matching node a little inside the forbidden region — harmless. A smaller box ($L = 6$) is sufficient because the quartic term makes the wavefunctions decay faster.

Exercise 9 — Finite square well

Question. Turn Problem 2 into a bound-state problem with $V = -V_0$ for $|x| < a/2$, using the shooting/matching method of Problem 3 and the discontinuity correction of Problem 2; verify the even-state condition $q\tan(qa/2) = \kappa$.

Analytical scheme. With $E < 0$, $q = \sqrt{2(V_0 - |E|)}$ inside and $\kappa = \sqrt{2|E|}$ outside, matching ψ'/ψ at $x = a/2$ gives

$$\text{even: } q\tan\frac{qa}{2} = \kappa, \qquad \text{odd: } -q\cot\frac{qa}{2} = \kappa.$$

With $V_0 = 10$, $a = 2$: $\sqrt{2V_0}\,a/2 = 4.47$, so there are $\lceil 4.47/(\pi/2)\rceil = 3$ bound states (even, odd, even). They were found with a root finder (`brentq`) to 10^{-15}.

Numerical scheme.

- Grid $[-8,8]$; the well edges $x = \pm 1$ are grid nodes with $V = -V_0/2$.
- Outward Numerov from -8 and inward from $+8$, matched at the **centre** $x = 0$ with the discrete Wronskian $D(E)$ of Problem 3. (The centre is inside the allowed region, and both integrations start in forbidden regions — the stable direction.)
- The edge correction of Problem 2 is applied in both directions: one-sided V in stencils neighbouring an edge, and at an edge node the term $\delta = -\frac{h^3}{12}[k^2]\psi'$, with ψ' from the side already integrated: $(3\psi_e - 4\psi_{e-1} + \psi_{e-2})/2h$ outward, $(-3\psi_e + 4\psi_{e+1} - \psi_{e+2})/2h$ inward.
- Odd states have $\psi(0) = 0$, so the two pieces are glued with the ratio at node $m+1$ instead of m.

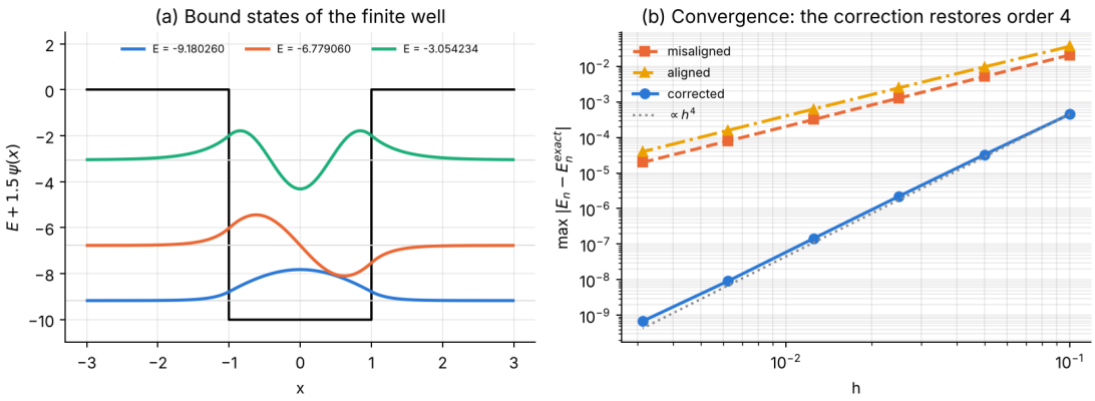
Results ($h = 0.01$, corrected):

state	E exact	E Numerov	|ΔE|	exact condition: lhs = κ	parity error
even (n=0)	−9.180259926233	−9.180259901486	2.5e-08	4.2849177183 = 4.2849177183	2.5e-14
odd (n=1)	−6.779060021410	−6.779059962042	5.9e-08	3.6821352559 = 3.6821352559	6.4e-10
even (n=2)	−3.054233508774	−3.054233501211	7.6e-09	2.4715313103 = 2.4715313103	1.2e-13

The exact energies satisfy the transcendental conditions to all printed digits; inserting the *numerical* energies leaves residuals of 3.7×10^{-7}, 2.3×10^{-7} and 1.5×10^{-8} — consistent with $|\Delta E|$. The numerical wavefunctions have the correct parity ($\psi(-x) = \pm\psi(x)$).

Convergence (largest $|\Delta E|$ among the three states):

h	misaligned	aligned $V_0/2$	corrected
0.1	2.09e-02	3.64e-02	4.51e-04
0.05	5.12e-03	9.66e-03	3.28e-05
0.025	1.28e-03	2.48e-03	2.21e-06
0.0125	3.18e-04	6.28e-04	1.44e-07
0.00625	7.96e-05	1.58e-04	9.15e-09
0.003125	1.99e-05	3.97e-05	6.80e-10
order	**2.01**	**1.96**	**3.90**



Exercise 9. (a) The three bound states of the finite well (even–odd–even); note the exponential tails in the classically forbidden regions. (b) Only the corrected treatment of the discontinuities converges with fourth order.

Answer. The method works and the exact even/odd conditions are verified. The behaviour seen for the barrier repeats exactly for the well: without the correction the energies converge only as h^2, with the correction as h^4 (at $h = 0.003$ the corrected scheme is 30 000–60 000 times more accurate).

Exercise 10 — Round-off in single precision

Question. Repeat the E_0 study of Problem 3 in single precision (Fortran `real(4)`) and locate the new optimum h.

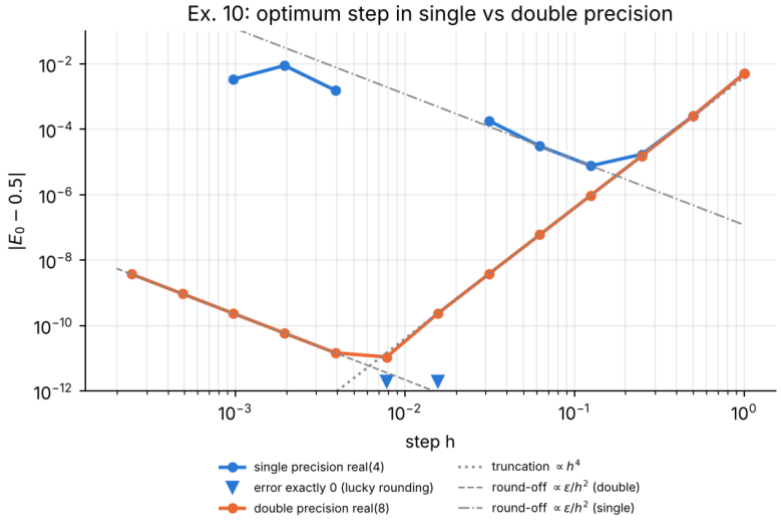
Theory (prediction). Total error $\approx Ah^4 + B\epsilon/h^2$. Minimising, $h_{\text{opt}} = (B\epsilon/2A)^{1/6} \propto \epsilon^{1/6}$ and $\varepsilon_{\min} \propto \epsilon^{2/3}$. With $\epsilon_{\text{single}}/\epsilon_{\text{double}} = 1.19 \times 10^{-7}/2.22 \times 10^{-16} = 5.4 \times 10^{8}$:

- h_{opt} should grow by $(5.4 \times 10^{8})^{1/6} = 28.5$;
- $\varepsilon_{\min}$ should grow by $(5.4 \times 10^{8})^{2/3} = 6.6 \times 10^{5}$.

Method. One Fortran source (`ex10_precision.F90`) with the working kind `wp` set by the preprocessor, compiled twice: `-DWP=4` (single) and `-DWP=8` (double). Two-sided Numerov, discrete-Wronskian matching, bisection carried down to machine precision, box $[-6,6]$.

Results.

N	h	$\lvert E_0-0.5\rvert$ single	$\lvert E_0-0.5\rvert$ double
12	1.0	4.96e-03	4.96e-03
24	0.5	2.52e-04	2.53e-04
48	0.25	1.72e-05	1.54e-05
96	**0.125**	**7.63e-06**	9.56e-07
192	0.0625	3.05e-05	5.96e-08
384	0.03125	1.83e-04	3.73e-09
768	0.015625	0 (lucky)	2.32e-10
1536	0.0078125	0 (lucky)	**1.09e-11**
3072	0.0039	1.53e-03	1.46e-11
6144	0.0020	8.85e-03	5.82e-11
12288	0.00098	3.39e-03	2.33e-10
24576	0.00049	fails (no sign change)	9.31e-10
49152	0.00024	fails	3.73e-09



Exercise 10. Error of E_0 in single and double precision. Right branches: truncation (h^4); left branches: round-off ($\propto \epsilon/h^2$). The two exact zeros of the single-precision run (triangles) are accidental.

Answer.

- **Single precision:** optimum $h \approx 0.125$ ($N = 96$, i.e. 8 points per unit length), best error 7.6×10^{-6}. Below $h \approx 0.004$ the matching function is pure noise and the bisection no longer even finds a sign change.
- **Double precision:** optimum $h \approx 0.008$, best error 1.1×10^{-11}; to the left of the optimum the error grows by a factor 4 per halving of h — exactly the ϵ/h^2 law.
- **Comparison with the prediction:** the measured ratio of optimal steps is $0.125/0.0078 = 16$ (predicted 28.5 — on a grid of factor-2 steps this is within one step), and the ratio of minimum errors is $7.6 \times 10^{-6}/1.1 \times 10^{-11} = 7 \times 10^5$ (predicted 6.6×10^5). The $\epsilon^{2/3}$ law is confirmed remarkably well.
- The two runs coincide for $h \geq 0.5$: there the truncation error dominates and precision is irrelevant.
- The two "exact" single-precision results ($E_0 = 0.5$ exactly at $N = 768{,}1536$) are accidents of rounding to the nearest representable float and must not be mistaken for convergence — a good reason to always look at the *trend* of the error, not individual values.

Practical lesson. In single precision Numerov cannot deliver more than about 5 significant digits, whatever the grid; refining beyond $h \approx 0.1$ makes the result *worse*. Scientific Numerov codes should always use double precision, and Richardson extrapolation from two moderately fine grids is preferable to brute-force refinement.

Appendix — Codes

All programs are in the accompanying zip archive. Run with

```
python3 ex01_05.py          # exercises 1-5   (needs numpy, matplotlib)
python3 ex06_09.py          # exercises 6-9   (needs scipy; imports ../python/p3_harmonic_oscillator.py)
python3 ex06_09.py 7 9      # only selected exercises
gfortran -O2 -cpp -DWP=4 -o ex10_sp ex10_precision.F90 && ./ex10_sp
gfortran -O2 -cpp -DWP=8 -o ex10_dp ex10_precision.F90 && ./ex10_dp
python3 ex10_plot.py        # plots the two .dat files
```

Exercises 1–5 (Python)

File: `ex01_05.py`

```
#!/usr/bin/env python3
"""Solutions of exercises 1-5 of the Numerov course (hbar = m = 1)."""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"]


def numerov(k2, p0, p1, h):
    c = 1.0 + h * h * k2 / 12.0
    p = np.zeros(len(k2), dtype=np.result_type(p0, p1, float))
    p[0], p[1] = p0, p1
    for n in range(1, len(k2) - 1):
        p[n + 1] = ((12.0 - 10.0 * c[n]) * p[n] - c[n - 1] * p[n - 1]) / c[n + 1]
    return p


def style(ax):
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")


# ============================================================== Ex 1
def ex1():
    print("\n### Exercise 1: order from two grids (Problem 1, Part A)")
    k, X = 2.0, 10.0
    Ns = [25, 50, 100, 200, 400, 800, 1600, 3200, 6400, 12800]
    e = []
    for N in Ns:
        h = X / N; x = np.arange(N + 1) * h
        p = numerov(np.full(N + 1, k * k), 0.0, np.sin(k * h), h)
        e.append(np.max(np.abs(p - np.sin(k * x))))
    print(f"{'N -> 2N':>14} {'eps(h)':>10} {'eps(h/2)':>10} {'p':>6}")
    for i in range(len(Ns) - 1):
        print(f"{Ns[i]:6d}->{Ns[i+1]:6d} {e[i]:10.3e} {e[i+1]:10.3e} {np.log2(e[i]/e[i+1]):6.2f}")
    # round-off estimate: eps_mach * N^2 (second-order recursion)
    for N, ee in zip(Ns, e):
        print(f"N={N:6d}  error={ee:.2e}   truncation est. {e[3]*(200/N)**4:.2e}   roundoff est. eps*N^2={2.2e-
16*N*N:.2e}")
    return Ns, e


# ============================================================== Ex 2
def ex2():
    print("\n### Exercise 2: third-order Taylor start psi_1 = kh - (kh)^3/6")
    k, X = 2.0, 10.0
    Ns = [25, 50, 100, 200, 400, 800, 1600, 3200]
    res = {"exact": [], "Taylor 1st": [], "Taylor 3rd": []}
    hs = []
    for N in Ns:
        h = X / N; x = np.arange(N + 1) * h; hs.append(h)
        for lab, p1 in (("exact", np.sin(k * h)), ("Taylor 1st", k * h),
                        ("Taylor 3rd", k * h - (k * h) ** 3 / 6)):
            p = numerov(np.full(N + 1, k * k), 0.0, p1, h)
            res[lab].append(np.max(np.abs(p - np.sin(k * x))))
    print(f"{'N':>6} {'h':>8} {'exact':>10} {'Taylor1':>10} {'Taylor3':>10} {'ratio3':>7}")
    for i, N in enumerate(Ns):
        r = res["Taylor 3rd"][i - 1] / res["Taylor 3rd"][i] if i else np.nan
        print(f"{N:6d} {hs[i]:8.5f} {res['exact'][i]:10.3e} {res['Taylor 1st'][i]:10.3e} "
              f"{res['Taylor 3rd'][i]:10.3e} {r:7.2f}")
    hs = np.array(hs)
```

```
    for lab in res:
        print(f"  order {lab}: {np.polyfit(np.log(hs[:6]), np.log(res[lab][:6]), 1)[0]:.2f}")
    # decomposition: Taylor-3 error minus exact-start error = start-value contribution
    d = np.abs(np.array(res["Taylor 3rd"]) - np.array(res["exact"]))
    print("  start-value contribution |e3-e_exact|:", " ".join(f"{v:.1e}" for v in d))
    fig, ax = plt.subplots(figsize=(6, 3.8))
    for i, (lab, st) in enumerate(zip(res, ("o-", "s-", "^--"))):
        ax.loglog(hs, res[lab], st, color=C[i], lw=2, ms=7 - 2 * i, label=f"start: {lab}")
    ax.loglog(hs, res["exact"][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax.loglog(hs, res["Taylor 1st"][0] * (hs / hs[0]) ** 2, "--", color="#888888", label=r"$\propto h^2$")
    ax.set(xlabel="h", ylabel=r"max $|\psi-\sin kx|$", title="Ex. 2: effect of the starting value")
    ax.legend(frameon=False, fontsize=8); style(ax)
    fig.tight_layout(); fig.savefig("ex2.png", dpi=170)


# ================================================================== Ex 3
def box_E(n, N, L=1.0):
    h = L / N
    f = lambda E: numerov(np.full(N + 1, 2 * E), 0.0, 1e-3 * h, h)[-1]
    Ex = (n * np.pi / L) ** 2 / 2
    a, b = Ex * (1 - 0.5 / n), Ex * (1 + 0.5 / n)          # brackets only the n-th zero
    fa = f(a)
    assert np.sign(fa) != np.sign(f(b))
    for _ in range(80):
        m = 0.5 * (a + b); fm = f(m)
        if np.sign(fm) == np.sign(fa): a, fa = m, fm
        else: b = m
    return 0.5 * (a + b), Ex


def ex3():
    print("\n### Exercise 3: smallest N giving E_10 with rel. error < 1e-6 (box L=1)")
    n = 10
    Ns, errs = [], []
    for N in range(20, 401, 2):
        E, Ex = box_E(n, N)
        Ns.append(N); errs.append(abs(E - Ex) / Ex)
    Ns, errs = np.array(Ns), np.array(errs)
    Nmin = Ns[np.argmax(errs < 1e-6)]
    lam = 2.0 / n
    print(f"  N_min = {Nmin}, rel.err = {errs[Ns == Nmin][0]:.3e}, previous N={Nmin-2}: {errs[Ns == Nmin-
2][0]:.3e}")
    print(f"  wavelength lambda = 2L/n = {lam}, points per wavelength = {Nmin*lam:.1f}, k h = {n*np.pi/Nmin:.4f}")
    # theory: rel error of E ~ (k h)^4 / 240 for the box (from the dispersion relation)
    for N in (40, 100, Nmin, 200):
        E, Ex = box_E(n, N); kh = n * np.pi / N
        print(f"  N={N:4d} kh={kh:.3f} rel.err={abs(E-Ex)/Ex:.3e}  (kh)^4/240={kh**4/240:.3e}")
    # other n: N needed for 1e-6
    print("  points per wavelength needed for 1e-6, other states:")
    for nn in (1, 2, 5, 10, 20):
        for N in range(max(8, 20 * nn), 4000, 2):
            E, Ex = box_E(nn, N)
            if abs(E - Ex) / Ex < 1e-6:
                print(f"    n={nn:3d}: N_min={N:5d}  points/wavelength={N*2/nn:.1f}")
                break
    fig, ax = plt.subplots(figsize=(6, 3.8))
    ppw = Ns * lam
    ax.loglog(ppw, errs, color=C[0], lw=2, label=r"Numerov, $E_{10}$")
    ax.loglog(ppw, (2 * np.pi / ppw) ** 4 / 240, "--", color=C[1], label=r"$(kh)^4/240$")
    ax.axhline(1e-6, color="#888888", ls=":"); ax.axvline(Nmin * lam, color="#888888", ls=":")
    ax.text(Nmin * lam * 1.04, 3e-3, f"{Nmin*lam:.0f} points\nper wavelength", fontsize=8)
    ax.set(xlabel="grid points per wavelength", ylabel="relative error of $E_{10}$",
           title="Ex. 3: resolution needed for $10^{-6}$")
    ax.legend(frameon=False, fontsize=8); style(ax)
    fig.tight_layout(); fig.savefig("ex3.png", dpi=170)


# ================================================================== general scattering solver
def scatter(E, h, Vfun, xl, xr, use_kh=False, edges=(), V0jump=None):
    """Backward Numerov for a general potential.  edges: x-positions of jumps
    (assumed on nodes); for those the corrected scheme of Problem 2 is used."""
    k = np.sqrt(2 * E)
    N = int(round((xr - xl) / h)); x = xl + h * np.arange(N + 1)
    V = Vfun(x)
    eidx = [int(round((xe - xl) / h)) for xe in edges]
    Vl = {i: Vfun(np.array([x[i] - 1e-9]))[0] for i in eidx}
    Vr = {i: Vfun(np.array([x[i] + 1e-9]))[0] for i in eidx}
    for i in eidx:
        V[i] = 0.5 * (Vl[i] + Vr[i])
    c = 1 + h * h * 2 * (E - V) / 12
    isedge = np.zeros(N + 1, bool); isedge[eidx] = True
    if use_kh:   # discrete wave number of the Numerov plane wave
```

```
        a = (k * h) ** 2
        kk = np.arccos((1 - 5 * a / 12) / (1 + a / 12)) / h
    else:
        kk = k
    psi = np.zeros(N + 1, complex)
    psi[N], psi[N - 1] = np.exp(1j * kk * x[N]), np.exp(1j * kk * x[N - 1])
    for n in range(N - 1, 0, -1):
        cp, cm = c[n + 1], c[n - 1]
        if not isedge[n]:
            if n + 1 < N + 1 and isedge[n + 1]: cp = 1 + h * h * 2 * (E - V[n]) / 12
            if isedge[n - 1]: cm = 1 + h * h * 2 * (E - V[n]) / 12
        rhs = (12 - 10 * c[n]) * psi[n] - cp * psi[n + 1]
        if isedge[n]:
            jump = 2 * (E - Vr[n]) - 2 * (E - Vl[n])
            dpsi = (-3 * psi[n] + 4 * psi[n + 1] - psi[n + 2]) / (2 * h)
            rhs += -h ** 3 / 12 * jump * dpsi
        psi[n - 1] = rhs / cm
    M = np.array([[np.exp(1j * kk * x[0]), np.exp(-1j * kk * x[0])],
                  [np.exp(1j * kk * x[1]), np.exp(-1j * kk * x[1])]])
    A, B = np.linalg.solve(M, psi[:2])
    # flux: incident current is proportional to the (discrete) velocity, identical on both sides
    return 1 / abs(A) ** 2, abs(B) ** 2 / abs(A) ** 2


def T_square(E, V0, a):
    kap = np.sqrt(2 * (V0 - E))
    return 1 / (1 + V0 ** 2 * np.sinh(kap * a) ** 2 / (4 * E * (V0 - E)))


# ================================================================== Ex 4
def ex4():
    print("\n### Exercise 4: plane waves e^{ikx} vs discrete e^{i k_h x}")
    V0, a = 5.0, 1.0
    Vf = lambda x: np.where((x > 0) & (x < a), V0, 0.0)
    hs = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125])
    out = {}
    for E in (2.0, 8.0):
        Tex = T_square(E, V0, a) if E < V0 else 1 / (1 + V0 ** 2 * np.sin(np.sqrt(2 * (E - V0)) * a) ** 2 / (4 * E
* (E - V0)))
        for pad in (2.0, 20.0):
            e1, e2 = [], []
            for h in hs:
                T1, _ = scatter(E, h, Vf, -pad, a + pad, False, edges=(0.0, a))
                T2, _ = scatter(E, h, Vf, -pad, a + pad, True, edges=(0.0, a))
                e1.append(abs(T1 - Tex)); e2.append(abs(T2 - Tex))
            out[E, pad] = (e1, e2)
            print(f" E={E}, padding={pad}:")
            for i, h in enumerate(hs):
                print(f"   h={h:8.5f}  |dT| e^ikx = {e1[i]:.3e}   e^ik_hx = {e2[i]:.3e}   ratio
{e1[i]/e2[i]:.2f}")
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8), sharey=True)
    for j, E in enumerate((2.0, 8.0)):
        for i, pad in enumerate((2.0, 20.0)):
            e1, e2 = out[E, pad]
            ax[j].loglog(hs, e1, "o-", color=C[2 * i], lw=2, label=f"$e^{{ikx}}$, padding {pad:g}")
            ax[j].loglog(hs, e2, "s--", color=C[2 * i + 1], lw=2, label=f"$e^{{ik_hx}}$, padding {pad:g}")
        ax[j].set(xlabel="h", title=f"E = {E}"); style(ax[j])
    ax[0].set_ylabel(r"$|T-T_{exact}|$"); ax[0].legend(frameon=False, fontsize=7)
    fig.suptitle("Ex. 4: continuum vs discrete plane waves at the boundaries", fontsize=10)
    fig.tight_layout(); fig.savefig("ex4.png", dpi=170)


# ================================================================== Ex 5
def ex5():
    print("\n### Exercise 5: thick barrier V0=5, a=6, E=1")
    V0, a, E = 5.0, 6.0, 1.0
    kap = np.sqrt(2 * (V0 - E)); k = np.sqrt(2 * E)
    Tex = T_square(E, V0, a)
    Twkb = np.exp(-2 * kap * a)
    Tasym = 16 * E * (V0 - E) / V0 ** 2 * np.exp(-2 * kap * a)
    print(f"  kappa={kap:.5f}, kappa*a={kap*a:.4f}")
    print(f"  T exact          = {Tex:.6e}")
    print(f"  T WKB e^-2ka     = {Twkb:.6e}   ratio exact/WKB = {Tex/Twkb:.4f}")
    print(f"  T asymptotic 16E(V0-E)/V0^2 e^-2ka = {Tasym:.6e}  (prefactor {16*E*(V0-E)/V0**2:.3f})")
    Vf = lambda x: np.where((x > 0) & (x < a), V0, 0.0)
    rows = []
    for h in (0.05, 0.025, 0.0125, 0.00625, 0.003125):
        T, R = scatter(E, h, Vf, -2.0, a + 2.0, edges=(0.0, a))
        rows.append((h, T, abs(T - Tex) / Tex, R, R + T - 1))
        print(f"  h={h:8.5f}  T={T:.8e}  rel.err={abs(T-Tex)/Tex:.2e}   R={R:.17f}  R+T-1={R+T-1:.2e}")
    # growth factor inside the barrier
    print(f"  |A| ~ 1/sqrt(T) = {1/np.sqrt(Tex):.3e}  (amplitude grows by e^(kappa a)={np.exp(kap*a):.2e} across
```

```
barrier)")
    # thickness scan
    print("  thickness scan, h=0.00625:")
    for aa in (1, 2, 4, 6, 8, 10, 12):
        Vf2 = lambda x, aa=aa: np.where((x > 0) & (x < aa), V0, 0.0)
        T, R = scatter(E, 0.00625, Vf2, -2.0, aa + 2.0, edges=(0.0, float(aa)))
        Te = T_square(E, V0, aa)
        print(f"    a={aa:3d}  T={T:.4e}  exact={Te:.4e}  rel.err={abs(T-Te)/Te:.2e}  WKB/exact={np.exp(-
2*kap*aa)/Te:.3f}  R+T-1={R+T-1:.1e}")


if __name__ == "__main__":
    ex1(); ex2(); ex3(); ex4(); ex5()
```

Exercises 6–9 (Python)

File: `ex06_09.py`

```
#!/usr/bin/env python3
"""Solutions of exercises 6-9 of the Numerov course (hbar = m = 1)."""
import sys
import numpy as np
from scipy.optimize import brentq
from scipy.linalg import eigh_tridiagonal, eigh
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

sys.path.insert(0, "../python")
import p3_harmonic_oscillator as P3          # re-use the course code of Problem 3

C = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4"]


def style(ax):
    ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")


def numerov(k2, p0, p1, h):
    c = 1.0 + h * h * k2 / 12.0
    p = np.zeros(len(k2)); p[0], p[1] = p0, p1
    for n in range(1, len(k2) - 1):
        p[n + 1] = ((12.0 - 10.0 * c[n]) * p[n] - c[n - 1] * p[n - 1]) / c[n + 1]
    return p


# ============================================================== Ex 6
def T_poschl(E, V0, w):
    k = np.sqrt(2 * E)
    s = 8 * V0 * w * w
    den = np.cosh(np.pi / 2 * np.sqrt(s - 1)) ** 2 if s > 1 else np.cos(np.pi / 2 * np.sqrt(1 - s)) ** 2
    return np.sinh(np.pi * k * w) ** 2 / (np.sinh(np.pi * k * w) ** 2 + den)


def scatter_smooth(E, h, Vf, X):
    k = np.sqrt(2 * E)
    N = int(round(2 * X / h)); x = -X + h * np.arange(N + 1)
    c = 1 + h * h * 2 * (E - Vf(x)) / 12
    psi = np.zeros(N + 1, complex)
    psi[N], psi[N - 1] = np.exp(1j * k * x[N]), np.exp(1j * k * x[N - 1])
    for n in range(N - 1, 0, -1):
        psi[n - 1] = ((12 - 10 * c[n]) * psi[n] - c[n + 1] * psi[n + 1]) / c[n - 1]
    M = np.array([[np.exp(1j * k * x[0]), np.exp(-1j * k * x[0])],
                  [np.exp(1j * k * x[1]), np.exp(-1j * k * x[1])]])
    A, B = np.linalg.solve(M, psi[:2])
    return 1 / abs(A) ** 2, abs(B) ** 2 / abs(A) ** 2


def ex6():
    print("\n### Exercise 6: smooth barrier V0/cosh^2(x/w), V0=5, w=0.5")
    V0, w, X = 5.0, 0.5, 12.0
    Vf = lambda x: V0 / np.cosh(x / w) ** 2
    print(f"  8 V0 w^2 = {8*V0*w*w}; V(+-{X}) = {Vf(X):.1e} (negligible)")
    for E in (1.0, 3.0, 5.0, 8.0):
        T, R = scatter_smooth(E, 0.01, Vf, X)
        print(f"  E={E}: T={T:.10f}  exact={T_poschl(E,V0,w):.10f}  |dT|={abs(T-T_poschl(E,V0,w)):.2e}  R+T-
1={R+T-1:.1e}")
    hs = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125])
    errs = {}
    for E in (2.0, 8.0):
```

```
        errs[E] = [abs(scatter_smooth(E, h, Vf, X)[0] - T_poschl(E, V0, w)) for h in hs]
        print(f"  E={E}: errors", " ".join(f"{e:.2e}" for e in errs[E]),
              f" ratios", " ".join(f"{errs[E][i]/errs[E][i+1]:.1f}" for i in range(len(hs) - 1)),
              f" order={np.polyfit(np.log(hs[:5]), np.log(errs[E][:5]), 1)[0]:.2f}")
    # square barrier of the same area for comparison (naive scheme, edges on nodes, V0/2)
    Eg = np.linspace(0.05, 15, 300)
    Tn = np.array([scatter_smooth(E, 0.01, Vf, X)[0] for E in Eg])
    Te = np.array([T_poschl(E, V0, w) for E in Eg])
    print(f"  max |dT| over 0<E<15 at h=0.01: {np.max(abs(Tn-Te)):.2e}")
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.7))
    ax[0].plot(Eg, Te, color="#888888", lw=1, label="exact (Pöschl-Teller)")
    ax[0].plot(Eg[::10], Tn[::10], "o", ms=4, color=C[0], label="Numerov h = 0.01")
    ax[0].set(xlabel="E", ylabel="T(E)", title=r"(a) $V=V_0/\cosh^2(x/w)$, $V_0=5$, $w=0.5$")
    ax[0].legend(frameon=False, fontsize=8)
    for i, E in enumerate((2.0, 8.0)):
        ax[1].loglog(hs, errs[E], "o-", color=C[i], lw=2, label=f"E = {E}")
    ax[1].loglog(hs, errs[8.0][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax[1].set(xlabel="h", ylabel=r"$|T-T_{exact}|$", title="(b) plain Numerov, no correction: order 4")
    ax[1].legend(frameon=False, fontsize=8)
    for a in ax: style(a)
    fig.tight_layout(); fig.savefig("ex6.png", dpi=170)


# ================================================================ Ex 7
def ex7():
    print("\n### Exercise 7: one-sided shooting for the oscillator ground state, h = 0.01")
    h = 0.01
    rows = []
    for L in (3, 4, 5, 6, 7, 8, 9, 10, 12, 15, 20, 30, 38):
        N = int(round(2 * L / h)); x = -L + h * np.arange(N + 1)
        def F(E):
            with np.errstate(all="ignore"):
                return numerov(2 * E - x * x, 0.0, 1e-10, h)[-1]
        a, b = 0.3, 0.7
        fa, fb = F(a), F(b)
        if not (np.isfinite(fa) and np.isfinite(fb)) or np.sign(fa) == np.sign(fb):
            rows.append((L, np.nan, "no sign change / overflow"))
            print(f"  L={L:3d}: psi_N(0.3)={fa:.2e}, psi_N(0.7)={fb:.2e} -> no usable bracket")
            continue
        for _ in range(100):
            m = 0.5 * (a + b); fm = F(m)
            if np.sign(fm) == np.sign(fa): a, fa = m, fm
            else: b = m
        E = 0.5 * (a + b)
        # sensitivity: how big is psi_N for dE = 1e-12 compared with its round-off noise?
        rows.append((L, E, ""))
        print(f"  L={L:3d}: E0 = {E:.15f}   |E0-0.5| = {abs(E-0.5):.2e}   e^(L^2/2)={np.exp(L*L/2):.1e}")
    # wavefunction obtained by one-sided shooting at the converged energy, L = 8
    L = 8.0; N = int(round(2 * L / h)); x = -L + h * np.arange(N + 1)
    E8 = [r[1] for r in rows if r[0] == 8][0]
    p = numerov(2 * E8 - x * x, 0.0, 1e-10, h)
    p /= np.max(abs(p[x < 0])) / np.pi ** -0.25          # scale so that psi(0) ~ exact
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
    Ls = [r[0] for r in rows if np.isfinite(r[1])]
    es = [max(abs(r[1] - 0.5), 1e-17) for r in rows if np.isfinite(r[1])]
    ax[0].semilogy(Ls, es, "o-", color=C[0], lw=2)
    ax[0].axhline(3.9e-11, color="#888888", ls=":")
    ax[0].text(9, 1e-10, "h-discretisation level (h = 0.01)", fontsize=8, color="#555555")
    ax[0].set(xlabel="L (box [-L, L])", ylabel=r"$|E_0-0.5|$", title="(a) One-sided shooting: eigenvalue error")
    ax[1].semilogy(x, np.abs(p) + 1e-300, color=C[1], lw=2, label="one-sided Numerov at converged $E_0$")
    ax[1].semilogy(x, np.pi ** -0.25 * np.exp(-x * x / 2), ":", color="k", lw=1.5, label=r"exact $\pi^{-1/4}e^{-
x^2/2}$")
    ax[1].set(ylim=(1e-16, 1e3), xlabel="x", ylabel=r"$|\psi(x)|$", title="(b) ...but the right tail is destroyed
(L = 8)")
    ax[1].legend(frameon=False, fontsize=8, loc="lower center")
    i_bad = np.where((x > 0) & (np.abs(p) > 10 * np.pi ** -0.25 * np.exp(-x * x / 2)))[0]
    print(f"  L=8: one-sided psi departs from exact (factor 10) beyond x = {x[i_bad[0]]:.2f}; psi(L) = {p[-
1]:.2e}")
    for a in ax: style(a)
    fig.tight_layout(); fig.savefig("ex7.png", dpi=170)


# ================================================================ Ex 8
def ho_basis_energies(lam, M=160):
    """Independent check: diagonalise H in the harmonic-oscillator basis."""
    n = np.arange(M + 4)
    X = np.diag(np.sqrt((n[1:]) / 2.0), 1); X = X + X.T
    X4 = np.linalg.matrix_power(X, 4)[:M, :M]
    H = np.diag(np.arange(M) + 0.5) + lam * X4
    return np.linalg.eigvalsh(H)
```

```python
def ex8():
    print("\n### Exercise 8: anharmonic oscillator V = x^2/2 + lambda x^4")
    orig = P3.V
    lam_list = [1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 0.1]
    table = {}
    for lam in lam_list:
        P3.V = lambda x, lam=lam: 0.5 * x * x + lam * x ** 4
        x, h, st = P3.eigenstates(nmax=3, L=6.0, N=1200, Emax=8.0)
        table[lam] = {n: E for n, E, _ in st}
    P3.V = orig
    print(f"{'lambda':>8} {'n':>2} {'E Numerov':>16} {'E basis':>16} {'dE/lambda':>11} {'1st-order coef':>14}
{'1st-order E':>12}")
    for lam in lam_list:
        Eb = ho_basis_energies(lam)
        for n in (0, 1, 2):
            E = table[lam][n]
            c1 = 0.75 * (2 * n * n + 2 * n + 1)
            print(f"{lam:8.0e} {n:2d} {E:16.12f} {Eb[n]:16.12f} {(E-n-0.5)/lam:11.6f} {c1:14.4f}
{n+0.5+c1*lam:12.6f}")
    lam = 0.1
    print(f"  lambda=0.1: E0 = {table[0.1][0]:.10f}; first order 0.575, 2nd order 0.575-21/8*0.01 = {0.575-
21/8*0.01:.6f},"
          f" 3rd order + 333/16*0.001 = {0.575-0.02625+333/16*0.001:.6f}")
    fig, ax = plt.subplots(figsize=(6.2, 3.9))
    lams = np.array(lam_list)
    for i, n in enumerate((0, 1, 2)):
        y = [(table[l][n] - n - 0.5) / l for l in lam_list]
        ax.semilogx(lams, y, "o-", color=C[i], lw=2, label=f"n={n} Numerov")
        ax.axhline(0.75 * (2 * n * n + 2 * n + 1), color=C[i], ls=":", lw=1)
    ax.semilogx(lams, 0.75 - 21 / 8 * lams, "--", color="#888888", lw=1, label="n=0: 3/4 − (21/8)λ")
    ax.set(xlabel=r"$\lambda$", ylabel=r"$(E_n-E_n^{(0)})/\lambda$",
           title="Ex. 8: shift / λ → first-order coefficient (dotted) as λ→0")
    ax.legend(frameon=False, fontsize=8); style(ax)
    fig.tight_layout(); fig.savefig("ex8.png", dpi=170)


# ================================================================ Ex 9
V0W, AW = 10.0, 2.0              # well depth and width


def well_exact():
    """Exact bound states from q tan(qa/2) = kappa (even), -q cot(qa/2) = kappa (odd)."""
    res = []
    f_even = lambda E: np.sqrt(2 * (V0W + E)) * np.tan(np.sqrt(2 * (V0W + E)) * AW / 2) - np.sqrt(-2 * E)
    f_odd = lambda E: -np.sqrt(2 * (V0W + E)) / np.tan(np.sqrt(2 * (V0W + E)) * AW / 2) - np.sqrt(-2 * E)
    Eg = np.linspace(-V0W + 1e-9, -1e-9, 200001)
    for f, par in ((f_even, "even"), (f_odd, "odd")):
        with np.errstate(all="ignore"):
            v = f(Eg)
        for i in np.where(np.sign(v[:-1]) != np.sign(v[1:]))[0]:
            if np.isfinite(v[i]) and np.isfinite(v[i + 1]) and abs(v[i]) < 50 and abs(v[i + 1]) < 50:
                res.append((brentq(f, Eg[i], Eg[i + 1], xtol=1e-15), par))
    return sorted(res)


def well_solve(h, mode="corrected", L=8.0, Emin=None):
    """Two-sided shooting for the finite well; mode in {'misaligned','aligned','corrected'}."""
    shift = 0.37 * h if mode == "misaligned" else 0.0
    N = int(round(2 * L / h)); x = -L + shift + h * np.arange(N + 1)
    inside = np.abs(x) < AW / 2
    V = np.where(inside, -V0W, 0.0)
    edges = []
    if mode != "misaligned":
        for xe in (-AW / 2, AW / 2):
            i = int(np.argmin(abs(x - xe))); V[i] = -V0W / 2; edges.append(i)
    Vl = {edges[0]: 0.0, edges[1]: -V0W} if edges else {}
    Vr = {edges[0]: -V0W, edges[1]: 0.0} if edges else {}
    isedge = np.zeros(N + 1, bool); isedge[edges] = True
    m = int(np.argmin(abs(x)))                    # match at the centre of the well
    corr = mode == "corrected"

    def shoot(E):
        k2 = 2 * (E - V); c = 1 + h * h * k2 / 12
        def cc(j, centre):                        # weight of node j as seen from stencil 'centre'
            if corr and isedge[j] and not isedge[centre]:
                return 1 + h * h * 2 * (E - V[centre]) / 12
            return c[j]
        pL = np.zeros(N + 1); pL[1] = 1e-10
        for n in range(1, m + 1):
            rhs = (12 - 10 * c[n]) * pL[n] - cc(n - 1, n) * pL[n - 1]
            if corr and isedge[n]:
                dpsi = (3 * pL[n] - 4 * pL[n - 1] + pL[n - 2]) / (2 * h)   # from the left (known) side
```

```
                rhs += -h ** 3 / 12 * (2 * (E - Vr[n]) - 2 * (E - Vl[n])) * dpsi
            pL[n + 1] = rhs / cc(n + 1, n)
        pR = np.zeros(N + 1); pR[N - 1] = 1e-10
        for n in range(N - 1, m - 1, -1):
            rhs = (12 - 10 * c[n]) * pR[n] - cc(n + 1, n) * pR[n + 1]
            if corr and isedge[n]:
                dpsi = (-3 * pR[n] + 4 * pR[n + 1] - pR[n + 2]) / (2 * h)  # from the right side
                rhs += -h ** 3 / 12 * (2 * (E - Vr[n]) - 2 * (E - Vl[n])) * dpsi
            pR[n - 1] = rhs / cc(n - 1, n)
        pL /= np.max(abs(pL[:m + 2])); pR /= np.max(abs(pR[m - 1:]))
        D = pL[m + 1] * pR[m] - pL[m] * pR[m + 1]
        psi = np.concatenate([pL[:m + 1], pR[m + 1:] * pL[m + 1] / pR[m + 1]])  # glue (pR[m] may vanish for odd
states)
        return D, psi

    Es = np.arange(-V0W + 0.01, -0.005, 0.01)
    Ds = np.array([shoot(E)[0] for E in Es])
    out = []
    for i in np.where(np.sign(Ds[:-1]) != np.sign(Ds[1:]))[0]:
        a, b, fa = Es[i], Es[i + 1], Ds[i]
        for _ in range(70):
            mm = 0.5 * (a + b); fm = shoot(mm)[0]
            if np.sign(fm) == np.sign(fa): a, fa = mm, fm
            else: b = mm
        E = 0.5 * (a + b)
        psi = shoot(E)[1]
        psi /= np.sqrt(np.sum(psi ** 2) * h)
        out.append((E, x, psi))
    return out


def ex9():
    print(f"\n### Exercise 9: finite square well V=-{V0W} for |x|<{AW/2}")
    ex = well_exact()
    for E, par in ex:
        q, kap = np.sqrt(2 * (V0W + E)), np.sqrt(-2 * E)
        lhs = q * np.tan(q * AW / 2) if par == "even" else -q / np.tan(q * AW / 2)
        print(f"  exact {par:4s}: E = {E:.12f}  q={q:.6f} kappa={kap:.6f}  check {lhs:.10f} = {kap:.10f}")
    st = well_solve(0.01)
    print("  Numerov, corrected scheme, h=0.01:")
    for (E, x, psi), (Ee, par) in zip(st, ex):
        q, kap = np.sqrt(2 * (V0W + E)), np.sqrt(-2 * E)
        lhs = q * np.tan(q * AW / 2) if par == "even" else -q / np.tan(q * AW / 2)
        print(f"   E = {E:.12f} ({par})  |dE| = {abs(E-Ee):.2e}   residual of exact condition {lhs-kap:.2e}"
              f"  parity check psi(x)={'+' if par=='even' else '-'}psi(-x): {np.max(abs(psi - (1 if par=='even'
else -1)*psi[::-1])):.1e}")
    hs = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625, 0.003125])
    errs = {m: [] for m in ("misaligned", "aligned", "corrected")}
    for h in hs:
        for mode in errs:
            r = well_solve(h, mode)
            errs[mode].append(max(abs(r[i][0] - ex[i][0]) for i in range(len(ex))))
    print(f"{'h':>9} {'misaligned':>12} {'aligned':>12} {'corrected':>12}   (max |dE| over all bound states)")
    for i, h in enumerate(hs):
        print(f"{h:9.5f} {errs['misaligned'][i]:12.2e} {errs['aligned'][i]:12.2e} {errs['corrected'][i]:12.2e}")
    for mm in errs:
        print(f"  order {mm}: {np.polyfit(np.log(hs[:5]), np.log(errs[mm][:5]), 1)[0]:.2f}")
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    xx = np.linspace(-3, 3, 7)
    ax[0].plot([-3, -1, -1, 1, 1, 3], [0, 0, -V0W, -V0W, 0, 0], color="k", lw=1.5)
    for i, (E, x, psi) in enumerate(st):
        sel = abs(x) < 3
        ax[0].axhline(E, color="#dddddd", lw=.8)
        ax[0].plot(x[sel], E + 1.5 * psi[sel], color=C[i], lw=2, label=f"E = {E:.6f}")
    ax[0].set(xlabel="x", ylabel=r"$E + 1.5\,\psi(x)$", title="(a) Bound states of the finite well")
    ax[0].set(ylim=(-11, 2.5))
    ax[0].legend(frameon=False, fontsize=7, loc="upper center", ncol=3)
    st_ = {"misaligned": ("s--", C[1]), "aligned": ("^-.", C[3]), "corrected": ("o-", C[0])}
    for mm in errs:
        ax[1].loglog(hs, errs[mm], st_[mm][0], color=st_[mm][1], lw=2, label=mm)
    ax[1].loglog(hs, errs["corrected"][0] * (hs / hs[0]) ** 4, ":", color="#888888", label=r"$\propto h^4$")
    ax[1].set(xlabel="h", ylabel=r"max $|E_n-E_n^{exact}|$", title="(b) Convergence: the correction restores order
4")
    ax[1].legend(frameon=False, fontsize=8)
    for a in ax: style(a)
    fig.tight_layout(); fig.savefig("ex9.png", dpi=170)


if __name__ == "__main__":
    which = sys.argv[1:] or ["6", "7", "8", "9"]
    for w in which:
        {"6": ex6, "7": ex7, "8": ex8, "9": ex9}[w]()
```

Exercise 10 (Fortran 90, single and double precision)

File: `ex10_precision.F90`

```
!=====================================================================
! Exercise 10 -- ROUND-OFF: harmonic-oscillator ground state E_0 in
! single precision (real(4)) versus double precision (real(8)).
! The same source is compiled twice with the C preprocessor:
!   gfortran -O2 -cpp -DWP=4 -o ex10_sp ex10_precision.F90
!   gfortran -O2 -cpp -DWP=8 -o ex10_dp ex10_precision.F90
! Method: two-sided Numerov shooting with the discrete-Wronskian
! matching function D(E) (identical to Problem 3), box [-L, L], L = 6.
! Output: h, |E_0 - 0.5| for N = 12 ... 76800  (file ex10_wpX.dat)
!=====================================================================
module prec
  implicit none
  integer, parameter :: wp = WP
end module prec

module ho_shoot
  use prec
  implicit none
contains
  function Dfun(E, L, N) result(D)
    real(wp), intent(in) :: E, L
    integer,  intent(in) :: N
    real(wp) :: D, h, x, c(0:N), pL(0:N), pR(0:N), sL, sR
    integer  :: j, m
    h = 2.0_wp*L/N
    do j = 0, N
       x = -L + j*h
       c(j) = 1.0_wp + h*h*(2.0_wp*E - x*x)/12.0_wp
    end do
    m = nint((sqrt(2.0_wp*E) + L)/h)          ! node at the right turning point
    m = max(2, min(m, N-2))
    pL = 0.0_wp; pR = 0.0_wp
    pL(1) = 1.0e-10_wp
    do j = 1, m
       pL(j+1) = ((12.0_wp - 10.0_wp*c(j))*pL(j) - c(j-1)*pL(j-1))/c(j+1)
    end do
    pR(N-1) = 1.0e-10_wp
    do j = N-1, m, -1
       pR(j-1) = ((12.0_wp - 10.0_wp*c(j))*pR(j) - c(j+1)*pR(j+1))/c(j-1)
    end do
    sL = maxval(abs(pL(0:m+1))); sR = maxval(abs(pR(m-1:N)))
    D = (pL(m+1)/sL)*(pR(m)/sR) - (pL(m)/sL)*(pR(m+1)/sR)
  end function Dfun

  function ground_state(L, N) result(E)
    real(wp), intent(in) :: L
    integer,  intent(in) :: N
    real(wp) :: E, a, b, fa, fm, mid
    integer  :: it
    a = 0.3_wp; b = 0.7_wp                    ! brackets only E_0 = 0.5
    fa = Dfun(a, L, N)
    do it = 1, 200                            ! bisection down to machine precision
       mid = 0.5_wp*(a + b)
       if (mid <= a .or. mid >= b) exit
       fm = Dfun(mid, L, N)
       if (sign(1.0_wp, fm) == sign(1.0_wp, fa)) then
          a = mid; fa = fm
       else
          b = mid
       end if
    end do
    E = 0.5_wp*(a + b)
  end function ground_state
end module ho_shoot

program ex10
  use prec
  use ho_shoot
  implicit none
  integer, parameter :: nN = 13
  integer  :: Ns(nN) = [12, 24, 48, 96, 192, 384, 768, 1536, 3072, 6144, 12288, 24576, 49152]
  real(wp) :: L, E, h
  integer  :: i
  character(len=20) :: fname
  L = 6.0_wp
  write(fname,'(a,i1,a)') 'ex10_wp', wp, '.dat'
  open(10, file=fname, status='replace')
```

```
  print '(a,i2,a,es10.3)', 'working precision: real(', wp, ')   epsilon = ', epsilon(1.0_wp)
  print '(a8,a12,a22,a14)', 'N', 'h', 'E_0', '|E_0 - 0.5|'
  do i = 1, nN
     h = 2.0_wp*L/Ns(i)
     E = ground_state(L, Ns(i))
     print '(i8,es12.4,f22.16,es14.4)', Ns(i), h, E, abs(E - 0.5_wp)
     write(10,'(2es16.7)') real(h,8), real(abs(E - 0.5_wp),8)
  end do
  close(10)
end program ex10
```

Exercise 10 — plotting script

File: `ex10_plot.py`

```
#!/usr/bin/env python3
"""Plot ex10_wp4.dat / ex10_wp8.dat written by ex10_sp and ex10_dp."""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
C = ["#2a78d6", "#eb6834"]
fig, ax = plt.subplots(figsize=(6.5, 4.6))
for i, (f, lab) in enumerate((("ex10_wp4.dat", "single precision real(4)"), ("ex10_wp8.dat", "double precision
real(8)"))):
    h, e = np.loadtxt(f, unpack=True)
    ee = np.where((e > 0) & (e < 0.1), e, np.nan)       # break the line at exact zeros / failures
    ax.loglog(h, ee, "o-", color=C[i], lw=2, ms=5, label=lab)
    z = e == 0
    if z.any():
        ax.plot(h[z], np.full(z.sum(), 2e-12), "v", color=C[i], ms=7, label="error exactly 0 (lucky rounding)")
hh = np.logspace(-3.7, 0, 50)
ax.loglog(hh, 9.6e-7 * (hh / 0.125) ** 4, ":", color="#888888", label=r"truncation $\propto h^4$")
ax.loglog(hh, 3.7e-9 * (hh / 2.44e-4) ** -2, "--", color="#888888", lw=1, label=r"round-off $\propto \epsilon/h^2$
(double)")
ax.loglog(hh, 3.05e-5 * (hh / 0.0625) ** -2, "-.", color="#888888", lw=1, label=r"round-off $\propto \epsilon/h^2$
(single)")
ax.set(ylim=(1e-12, 1e-1), xlabel="step h", ylabel=r"$|E_0-0.5|$",
       title="Ex. 10: optimum step in single vs double precision")
ax.legend(frameon=False, fontsize=7, loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=2)
ax.spines[["top", "right"]].set_visible(False); ax.grid(alpha=.25, which="both")
fig.tight_layout(); fig.savefig("ex10.png", dpi=170)
```